From hand-rolled boosting to the libraries that win

In **[19]** you built gradient boosting yourself: fit each tree to the *negative gradient* of the loss, shrink, repeat. The algorithm is finished.

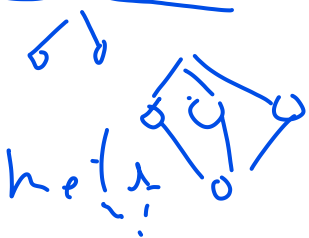
What is left is **engineering** - the libraries that dominate tabular ML:

- **XGBoost** - a regularized objective + a fast, careful implementation;
- **LightGBM** - leaf-wise growth, smart sampling, blazing speed;
- **CatBoost** - categorical features done right.

Same gradient descent in function space - made **fast**, **regularized**, and **production-ready**.

Outline

XGBoost: the regularized objective and the structure score
Engineering: why it is fast
The modern trio: LightGBM and CatBoost
Using them well
Stacking: combining different models



LightGBM ♥: էլ ուրիշ մոդելի անուն չգիտե՞ս

▶ watch on YouTube

XGBoost

Extreme gradient boosting: a regularized objective + a closed-form structure score.

2016 · Chen & Guestrin (Univ. of Washington / DMLC) · `github.com/dmlc/xgboost`

XGBoost: extreme gradient boosting

A scalable tree-boosting system (Chen & Guestrin, 2016):

- a **regularized objective** - penalties baked into the loss;
- **approximate / histogram** split finding for large data;
- row + column **subsampling**; **sparsity-aware** splits;
- parallel split search and **GPU** support.

Often the top performer on tabular benchmarks - **if properly tuned**. Many knobs, and the defaults are rarely optimal.

The plan, in plain words

Before any algebra, here is the whole idea:

1. **Score** how good a tree is by how much it would cut the loss - call it the *structure score*.
2. To get that score, **approximate the loss as a parabola** at the current prediction (a Newton step: use the slope *and* the curvature).
3. **Solve** that parabola for the best value in each leaf.
4. **Grow greedily**: keep a split only if it raises the score by more than the cost of adding a leaf.

The next slides just make each step precise. Hold on to this picture: **score a tree, then split to improve the score**.

A regularized objective

At step m we add one tree b with leaf values $c_1, \dots, c_T$. XGBoost minimizes the loss *plus* complexity penalties:

$$\text{obj} = \sum_{i=1}^{n} L\big(y_i,\, f^{[m-1]}(x_i) + b(x_i)\big) + \underbrace{\gamma T}_{\#\text{ leaves}} + \underbrace{\tfrac{1}{2}\lambda \|c\|_2^2}_{L_2} + \underbrace{\alpha \|c\|_1}_{L_1}.$$

- γ penalizes the **number of leaves** (tree size);
- λ (L_2) and α (L_1) shrink the **leaf values**.

Contrast [19]: plain gradient boosting put *no* penalty inside the objective. Here regularization is part of what every tree optimizes. (Software: `gamma`, `lambda`, `alpha`.)

Second-order Taylor of the loss

Expand the loss around the current prediction $f^{[m-1]}$, to **second order**:

$$L\big(y,\, f^{[m-1]} + b\big) \;\approx\; L\big(y,\, f^{[m-1]}\big) + g\, b + \tfrac{1}{2} h\, b^2,$$

with gradient and Hessian

$$g = \frac{\partial L}{\partial f^{[m-1]}}, \qquad h = \frac{\partial^2 L}{\partial (f^{[m-1]})^2}.$$

g is exactly the **negative pseudo-residual** from [19]. XGBoost's one real addition is the **curvature** h. *Intuition:* picture a parabola hugging the loss - slope g points downhill, bend h says how big a step to trust (a Newton step).

Group the objective by leaf

Drop the constant $L(y, f^{[m-1]})$ and use that every point in leaf t shares one value c_t:

$$\text{obj} \approx \sum_{t=1}^{T} \left[G_t\, c_t + \tfrac{1}{2}(H_t + \lambda)\, c_t^2 \right] + \gamma\, T,$$

$$G_t = \sum_{i \in \text{leaf } t} g_i, \qquad H_t = \sum_{i \in \text{leaf } t} h_i.$$

Intuition: every point in a leaf gets the **same** correction c_t, so a leaf is just **one number to choose** - and the objective is now a simple quadratic in it.

Optimal leaf value = the "structure score"

Minimizing each quadratic $G_t c_t + \frac{1}{2}(H_t + \lambda)c_t^2$ gives the best leaf value

$$c_t^* = -\frac{G_t}{H_t + \lambda}.$$

Plugging back, the best objective for a *fixed tree shape* is the **structure score**

$$\text{obj}^* = -\tfrac{1}{2}\sum_{t=1}^{T} \frac{G_t^2}{H_t + \lambda} + \gamma\, T.$$

Intuition: a leaf's best step = total error to fix (G_t) per unit of confidence ($H_t+\lambda$); the score adds up each leaf's payoff - **lower is a better tree**.

Numbers: $G_t = -3,\ H_t = 4,\ \lambda = 1 \ \Rightarrow\ c_t^* = -(-3)/(4+1) = 0.6.$

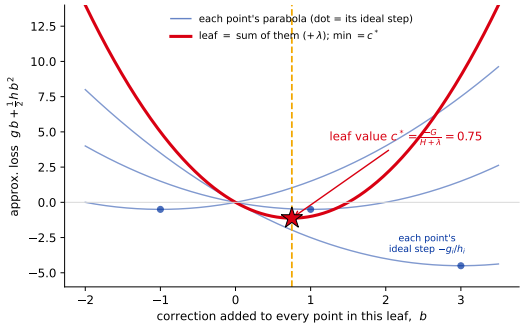
So how does the parabola become a tree?

The Taylor step gave **each point** its own parabola in the correction b; its ideal step is the Newton point $-g_i/h_i$ (the blue dots).

But a tree **cannot** give each point its own value – everyone in a leaf shares *one* number c_t. So we **add the leaf's parabolas** and jump to the bottom of the sum:

$$c_t^* = -\frac{G_t}{H_t + \lambda},$$

the curvature-weighted **compromise** of what its points want (here 0.75).



So what do the **splits** do? They **group points that want similar steps into the same leaf**, so each leaf's compromise is tight. The split-gain score (next) rewards exactly that grouping – the tree structure just decides *who shares a parabola*.

Worked bridge: the leaf value is [19]'s residual mean, braked

In [19] the round-1 **left leaf** just used the residual mean, -17.1. XGBoost's $c^* = -G_t/(H_t + \lambda)$ turns out to give the *same* leaf, gently shrunk by λ.

For **squared loss** the Hessian is $h_i = 1$, so a leaf with n rows has

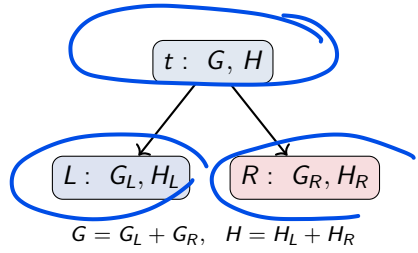
$$G_t = \sum_{\text{leaf}} -(y_i - F) = -\sum r_i, \qquad H_t = n \;\Rightarrow\; c_t^* = \frac{\sum r_i}{n + \lambda} = \underbrace{\bar{r}}_{\text{[19]'s leaf}} \cdot \frac{n}{n + \lambda}.$$

Left leaf: $n = 41$, $\bar{r} = -17.1$, so $c^* = -17.1 \times \frac{41}{42} = \mathbf{-16.7}$ at $\lambda = 1$ (barely braked), or -13.8 at $\lambda = 10$. **The structure score is [19]'s by-hand step plus a built-in λ brake.**

The split-gain criterion

How good is a split? By how much it **improves the structure score**:

$$\text{gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\right] - \gamma.$$



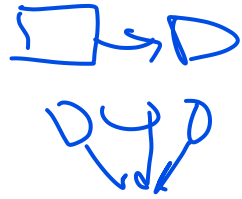
- *Intuition:* do the two children together score better than the parent, by more than the cost γ of a new leaf?
- Take the split only if $\text{gain} > 0$; γ is the **minimum gain to keep it**.
- This - not Gini/variance - is what XGBoost maximizes at each node.

Numbers: parent ($G{=}-3, H{=}4$), $\lambda{=}1$, split into $L(-3,2)$, $R(0,2)$: $\text{gain} = \frac{1}{2}[\frac{9}{3} + \frac{0}{3} - \frac{9}{5}] - \gamma = 0.6 - \gamma$ (keep only if $\gamma < 0.6$); then $c_L^*{=}1.0$, $c_R^*{=}0$.

Why it is fast

The same gradient boosting, engineered for speed and scale.

Tree growing and pruning



- Grow each tree **level by level**, up to `max_depth`.
- A split is kept only if its **gain** (previous slide) is positive.
- After growing, **prune** back any split with $\text{gain} < 0$ - the γ threshold.

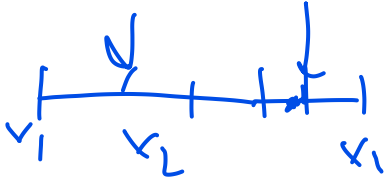
Depth-limited growth + post-pruning by gain keeps trees shallow and regularized.

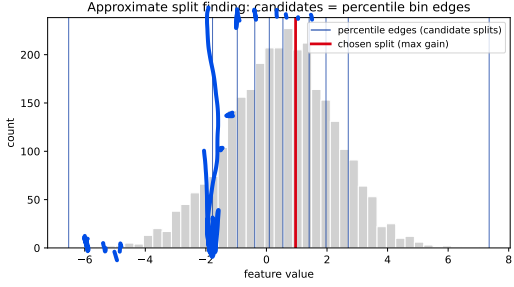
Approximate (histogram) split finding

Testing every possible split is slow. Instead, bucket each feature into $\sim \ell$ **percentile bins** and evaluate only the **bin edges**.

- **Global**: propose the bins once per tree.
- **Local**: re-propose at every node (more accurate, more work).

"Histogram-based gradient boosting" - XGBoost `tree_method="hist"`; the default in LightGBM and sklearn `HistGradientBoosting`.





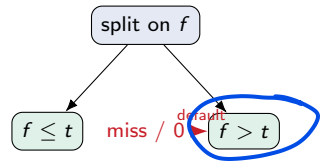
Candidates sit at percentile edges (denser where data is dense). Source: Chen & Guestrin (2016).

Sparsity-aware splits

Real data is full of **missing values** and **zeros** (one-hot columns).

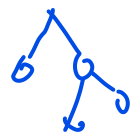
XGBoost gives every split a learned **default direction**: rows missing (or zero) on the split feature all go that way.

- Handles missing values **natively** - no imputation.
- Skips absent entries during split search, so sparse data is **faster**.



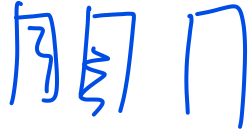
A learned default branch for absent values.

Subsampling, parallelism, GPU



- **Row subsampling** (`subsample`) - stochastic gradient boosting.
- **Column subsampling** (`colsample_bytree/bylevel/bynode`) - like a random forest's `mtry`; faster and more diverse trees.
- Boosting is **sequential**, but the **split search inside one tree** parallelizes over sorted feature blocks - and moves well to **GPU**.

Subsampling is both a speed-up and a regularizer.



DART: dropout for trees

DART (*Dropouts meet Multiple Additive Regression Trees*) borrows **dropout** from deep learning. When adding a new tree, compute the update against a **random subset** of the existing trees (drop the rest), then rescale so the ensemble does not overshoot.

- $p_{\text{drop}} = 0$: ordinary gradient boosting.
- $p_{\text{drop}} = 1$: trees trained independently, equally weighted $\approx$ a random forest.

p_{drop} is a smooth dial from **boosting** to **random forest** (XGBoost `booster="dart"`).

The modern trio

XGBoost is the baseline. LightGBM (the favorite) and CatBoost push it further.

LightGBM ♡

Leaf-wise growth, gradient sampling (GOSS), and feature bundling (EFB) – blazing speed.

2017 · Ke et al. (Microsoft) · `github.com/microsoft/LightGBM`

LightGBM ♡

For everyday tabular work, a great first thing to reach for:

- **very fast** and **low memory** (histogram + leaf-wise);
- excellent **out-of-the-box defaults**;
- **native categorical** support;
- scales smoothly to **large** data.

A solid **default** first model on tabular data – the next frames are why.

Predict-first: leaf-wise vs level-wise

Give both the **same budget** of leaves. Level-wise grows a full depth level at a time; leaf-wise always splits the *highest-loss* leaf. Which **overfits sooner**?

Predict-first: leaf-wise vs level-wise

Give both the **same budget** of leaves. Level-wise grows a full depth level at a time; leaf-wise always splits the *highest-loss* leaf. Which **overfits sooner**?

Leaf-wise. Spending the whole leaf budget on the worst region makes *deeper, narrower* trees that can chase noise – which is exactly why LightGBM needs `num_leaves` (and `max_depth`) kept in check.

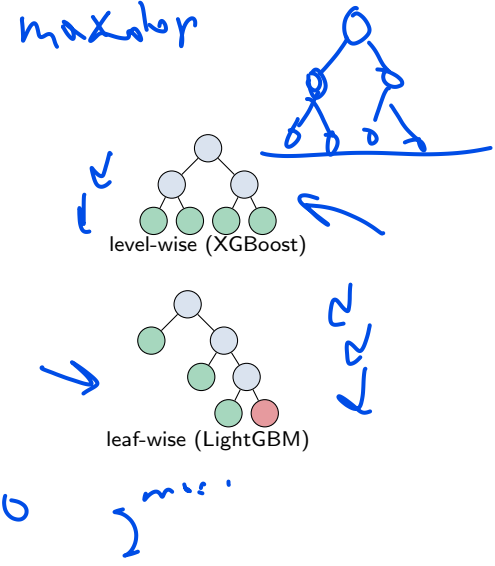
Leaf-wise (best-first) growth

XGBoost grows **level-wise by default** (a whole depth level at a time). LightGBM instead splits the **single leaf with the largest gain**, anywhere in the tree.

- Unbalanced, deeper trees; **converges in fewer rounds**.
- Can **overfit** - so the main complexity knob is `num_leaves` (keep it $< 2^{\texttt{max\_depth}}$), plus `min_data_in_leaf`.

Intuition: spend each split where it helps most, instead of splitting every leaf just because it is at the same depth.

Source: LightGBM docs.



Predict-first: GOSS throws away rows

Gradient-based One-Side Sampling (GOSS) keeps all **large-gradient** rows but **randomly drops most small-gradient** ones (the points the model already fits well). How much accuracy do you lose?

Predict-first: GOSS throws away rows

Gradient-based One-Side Sampling (GOSS) keeps all **large-gradient** rows but **randomly drops most small-gradient** ones (the points the model already fits well). How much accuracy do you lose?

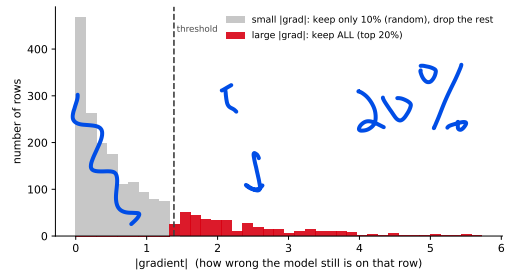
Almost none. Well-fit rows carry little new signal, so dropping most of them (with a reweight to stay unbiased) speeds training a lot at negligible cost – that is the whole point.

GOSS, part 1: what "gradient" means here

The "gradient" is nothing new – it is the **per-row gradient** $g_i = \partial L / \partial F$ from [19], at the current model:

- $|g_i|$ = **how wrong the model still is** on row i.
- For **squared loss** $g_i = -(y_i - F_i)$, so $|g_i|$ is just the **residual size**; for log-loss it is $|p_i - y_i|$.
- **Big** $|g_i|$: a still-misfit, *informative* row. **Small** $|g_i|$: already learned, little left to teach.

So sample rows *by* $|g_i|$ instead of uniformly – keep the hard ones.



Most rows have a small |gradient| (already fit); a few are still hard.

GOSS, part 2: the recipe

Gradient-based One-Side Sampling keeps the informative rows and thins the rest:

1. **Keep** the top $a \cdot n$ rows with the **largest** $|g_i|$ (the still-hard cases).
2. **Randomly sample** $b \cdot n$ of the small-gradient rest.
3. **Up-weight** that sample by $\frac{1-a}{b}$ so the split-gain estimate stays **unbiased** (the kept easy rows now stand in for all of them).

Defaults $a = 0.2$, $b = 0.1$: e.g. **1000 rows** $\rightarrow$ keep the 200 hardest + a random 80 of the other 800 = train on $\sim$280.

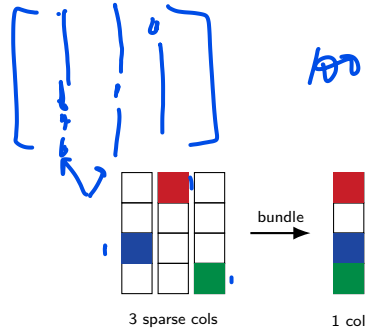
Why it works: rows the model already gets right carry little new signal – spend the budget on the hard rows, and reweight the easy sample so the data still looks unbiased. Big speed-up, negligible accuracy loss. Ke et al.

EFB: exclusive feature bundling

In sparse, high-dimensional data many features are **mutually exclusive** - never nonzero at the same time (one-hot columns).

Bundle them into one feature with a **single histogram** - nearly lossless.

- Histogram cost $\mathcal{O}(n\,p) \to \mathcal{O}(n \cdot \#\text{bundles})$.
- Optimal bundling is NP-hard; a **greedy** grouping works well.



Intuition: one-hot columns rarely fire together - pack them into one and lose almost nothing while doing far less work.

Ke et al. (2017).

EFB by example: how two features actually merge

Two sparse features never nonzero together (two one-hot columns), $A \in \{0 \dots 10\}$, $B \in \{0 \dots 5\}$. **Shift B by an offset** $= \max(A) = 10$:

$$\text{bundle} = \begin{cases} A & A > 0 \\ B + 10 & B > 0 \\ 0 & \text{both } 0 \end{cases}$$

A	B	bundle
3	0	3
0	2	**12**
0	0	0
0	4	**14**

Read it back: bundle $\leq 10 \Rightarrow A$'s value; bundle $> 10 \Rightarrow B$'s value (-10).

One histogram now holds *both* in **disjoint bins**, so a split on the bundle *is* a split on one original feature – **nothing lost**.

Exclusivity is the trick: A, B never both nonzero, so each value decodes unambiguously. (Real EFB allows a *few* conflicts to bundle more.)

CatBoost

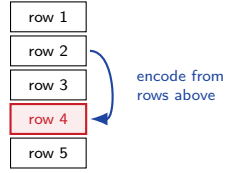
Ordered target statistics and ordered boosting – categorical features without target leakage.

2018 · Prokhorenkova et al. (Yandex) · `github.com/catboost/catboost`

CatBoost: the categorical specialist

Built around **categorical** features:

- **Ordered target statistics** - encode a category by its target mean on *past* rows only, so **no target leakage**;
- **Ordered boosting** - a permutation trick that removes the **prediction-shift** bias;
- **Symmetric (oblivious) trees** - one split per level: fast, regularized.



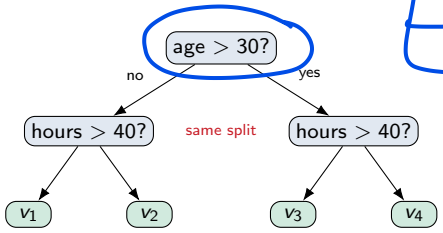
Intuition: the whole-dataset average peeks at the answer (leakage); a running average over *earlier* rows is honest - that is what "ordered" means.

CatBoost's oblivious (symmetric) trees

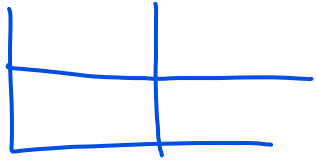
An **oblivious** tree uses the **same split on every node at a given depth** – the whole level tests one (feature, threshold).

- A depth-d tree is just d **conditions**; its 2^d leaves are indexed by the d yes/no answers (a bit-vector).
- **Very fast** prediction: compute d tests $\rightarrow$ look the leaf up in an array (no node-by-node traversal).
- The forced symmetry is a **regularizer** – balanced trees, less overfitting.

A normal CART tree may split each node on a *different* feature; oblivious trees give that up for speed and regularity.



Both level-2 nodes ask the *same* question.



Leaf = a lookup by (age>30, hours>40) – four cells, like a tiny decision **table**. That table structure is what makes CatBoost inference so fast.

Which one, when?

	XGBoost	**LightGBM**	**CatBoost**
Tree growth	level-wise	leaf-wise	symmetric
Row sampling (option)	random	random or GOSS	random
Native categoricals	no*	yes	yes (best)
Speed on large n	good	**fastest**	good
Reach for it when	solid baseline	default for tabular	many categoricals

* XGBoost can also grow leaf-wise and gradient-subsample, and has experimental categorical support. The table shows *defaults* - except row sampling, which is *off* by default in all three (cells name the available strategy). **All three are the same gradient boosting** - the differences are engineering and categorical handling.

Using them well

Tuning order, domain constraints, and reading the model.

Hyperparameters that matter

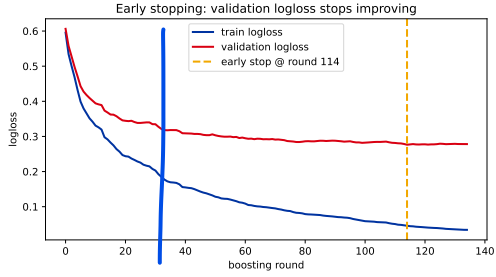
Group	XGBoost	LightGBM
Learning	`eta`, `n_estimators` (+ early stop)	`learning_rate`, `n_estimators`
Complexity	`max_depth`, `min_child_weight`	`num_leaves`, `min_data_in_leaf`
Regularize	`gamma`, `lambda`, `alpha`	`min_gain_to_split`, `lambda_l1/l2`
Randomness	`subsample`, `colsample_by*`	`bagging_fraction`, `feature_fraction`

The knob differs: XGBoost sets tree size with `max_depth`; LightGBM with `num_leaves` (default 31). Defaults: XGB `eta` 0.3 / depth 6; LGBM lr 0.1.

The tuning order that matters

1. Set `n_estimators` *large* and use **early stopping**.
2. Trade **learning rate** against the number of trees.
3. **Tree complexity**: `max_depth` (XGB) / `num_leaves` (LGBM).
4. **Regularization**: `gamma`, `lambda`, `alpha`.
5. **Subsampling**: rows + columns.

Search with random or Bayesian optimization (see the hyperparameter-tuning deck).

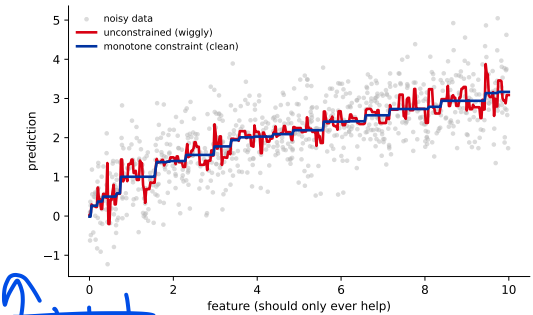


Early stopping takes the round before validation stalls - step 1.

Inject domain knowledge: constraints

Sometimes you *know* something the data should obey:

- **Monotonic constraints** (`monotone_constraints`): force a feature's effect to go only one way. E.g. *bigger apartment* ⇒ *predicted rent must not drop*; *higher income* ⇒ *credit score must not drop*.
- **Interaction constraints**: choose which features may combine in a tree. E.g. let `area` and `district` interact (a big flat *downtown* is special), but make `floor` act on its own.



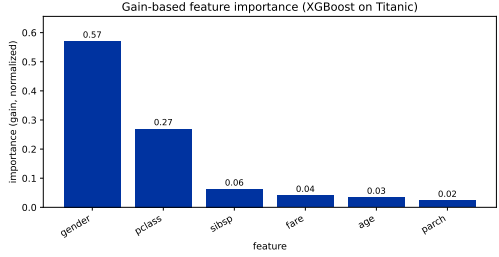
Constraints add **trust** and act as **free regularization** - use them when the direction or structure is known a priori.

Are they black boxes? Feature importance

Boosted models are **not** opaque:

- built-in importances: **gain** (loss reduction), cover, frequency;
- but **split-count importance is biased** toward high-cardinality features - prefer **permutation importance**.

For per-prediction attribution there is **SHAP** (fast *TreeSHAP* for trees) - we will cover it in its own session.



In code: LightGBM and XGBoost, side by side

LightGBM (the favorite)

```
from lightgbm import LGBMClassifier
from lightgbm import early_stopping
m = LGBMClassifier(
    n_estimators=2000,
    learning_rate=0.05,
    num_leaves=31)
m.fit(X_tr, y_tr,
  eval_set=[(X_val, y_val)],
  callbacks=[early_stopping(50)])
```

XGBoost (near-identical API)

```
from xgboost import XGBClassifier
m = XGBClassifier(
    n_estimators=2000,
    learning_rate=0.05,
    max_depth=6,
    early_stopping_rounds=50)
m.fit(X_tr, y_tr,
  eval_set=[(X_val, y_val)])
```

No install? `sklearn.ensemble.HistGradientBoostingClassifier` mirrors both.

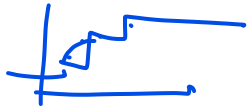
Beyond binary classification

The same g, h machinery powers many tasks - just change the **objective**:

- **Regression** (`reg:squarederror`), **multiclass** (softmax), **ranking** (`rank:*` / LambdaMART), Poisson, quantile - even **custom** losses (supply your own g and h).

Imbalanced classes? Reweight the rare class with `scale_pos_weight` (XGBoost) or `is_unbalance` / `class_weight` (LightGBM), and judge with **AUC / PR**, not accuracy. Full playbook: the imbalanced-learning lecture ([15]).

When boosting struggles



Powerful, but not magic:

- **No extrapolation** - a tree predicts a constant outside the training range; it cannot trend beyond what it has seen.
- **Needs tuning** - strong defaults exist, but top results need real HP search (a random forest is more forgiving).
- **Sequential** - trees are built one after another; less parallel than bagging.
- **No online learning** - batch-trained; you cannot cheaply update it example-by-example (as SGD models can). New data usually means **retraining**.
- **Not for raw unstructured data** - images, audio, long text → neural nets; very sparse high-dimensional signals can favor linear models.
- **Miscalibrated scores** - boosted `predict_proba` is often over-confident; calibrate before you trust it (calibration lecture, [14]).

Rule of thumb: reach for boosting on **tabular** data; reach elsewhere when the data is not tabular.

Stacking

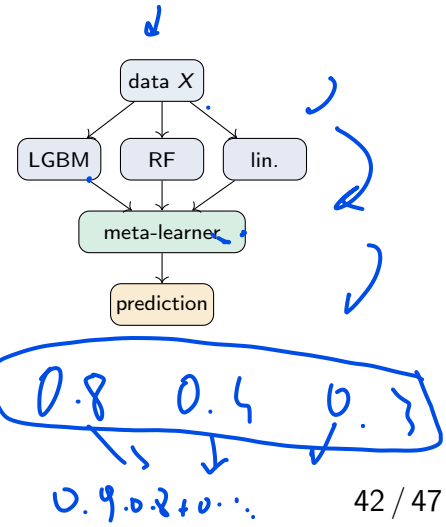
The third ensemble idea: let a meta-learner blend different model types - and close the chapter.

Stacking: a meta-learner over base models

Bagging and boosting combine *one* kind of model. **Stacking** combines **different** ones:

- train several diverse **base learners** (level 0) - e.g. LightGBM, a random forest, a linear model;
- feed their predictions to a **meta-learner** (level 1) that learns how best to *combine* them.

Intuition: different models make different mistakes; the meta-learner learns whom to trust, and when.



Stacking done right: out-of-fold predictions

The trap: if the meta-learner trains on base predictions of the *same* rows the base models were fit on, it sees **leaked**, over-optimistic inputs.

The fix: feed it **out-of-fold** (cross-validated) predictions - each row's base prediction comes from a model that did *not* train on it. `StackingClassifier` does this for you via `cv`.

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression

base = [("lgbm", LGBMClassifier()), ("rf", RandomForestClassifier())]
clf = StackingClassifier(estimators=base,
                         final_estimator=LogisticRegression(), cv=5)
clf.fit(X_tr, y_tr)   # cv=5 -> honest out-of-fold base predictions
```

Simpler cousins: voting and blending

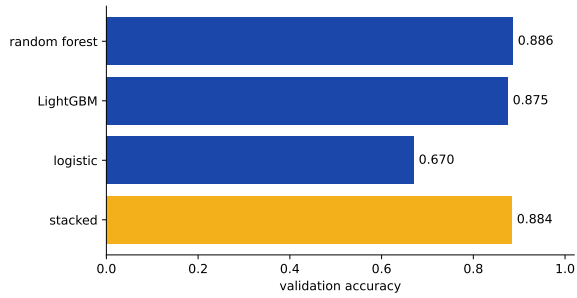
- **Voting** - no meta-learner, just combine base predictions directly: **hard** (majority class) or **soft** (average predicted probabilities, usually better). `VotingClassifier`.
- **Blending** - like stacking, but the meta-learner trains on a single **holdout** set instead of full cross-validation. Simpler and faster; uses a bit less data.

Stacking usually wins but costs the most (every base model + the meta-learner, with CV). Start with one strong model; stack only to chase the last bit of accuracy.

Does stacking actually pay off?

A quick receipt on one dataset: three diverse bases, then a logistic meta-learner over their out-of-fold predictions.

Here the stack **ties** the best base (random forest) and does not beat it – the weak logistic base added no usable diversity.



The honest lesson: stacking's gains are **real but small and not guaranteed**. Start with one strong model; reach for a stack only when the bases are **diverse and each genuinely good**.

The ensemble landscape

	Bagging (random forest)	**Boosting** (GB / XGB / LGBM)	**Stacking**
Base models	same type, parallel	same type, sequential	different types
Combine by	averaging	weighted additive sum	a meta-learner
Mainly cuts	variance	bias	both
Trains	in parallel	sequentially	bases parallel, then meta
Use when	low-tuning baseline	top tabular accuracy	squeeze diverse models

Three ways to turn many so-so models into one strong one.

Recap: the trees and ensembles chapter

- **One tree** ([17]) - interpretable, but unstable.
- **Bagging** $\rightarrow$ **random forests** ([18]) - average many parallel trees, cut variance.
- **Boosting** ([19]) - add trees sequentially to fix errors, cut bias; gradient descent in function space.
- **The libraries** ([20]) - XGBoost (regularized objective + structure score), **LightGBM** ♡ (leaf-wise + GOSS + EFB), CatBoost (categoricals); tune in order, add constraints, read importance.
- **Stacking** - a meta-learner over diverse models, to squeeze out the last gains.

Trees + ensembles dominate tabular data. Next: models for data where trees struggle - images, text, sequences.